

FuturesPro

期货量化交易系统

完整实现方案

版本: v1.0
日期: 2026-04-26
状态: 方案设计阶段

1 项目概述

1.1 项目背景

FuturesPro 是一套专为专业期货交易交易者打造的 AI 驱动量化交易平台。系统融合了实时行情分析、智能交易信号识别、策略回测与管理、风险控制等核心功能，旨在提供一个从研究到交易的全流程量化解决方案。

1.2 核心目标

- 实现一套完整的期货量化交易平台，涵盖仪表盘、实时交易、回测系统、交易雷达、策略管理、交易历史、账户中心 7 大模块
- 提供实时行情推送、K 线图表、订单簿等专业交易工具
- 集成 AI 智能交易信号识别、市场情绪分析、形态检测等能力
- 支持策略回测、参数优化、风险分析等量化研究功能
- 实现完善的风控体系和账户安全管理

1.3 系统页面清单

序号	页面名称	英文标识	核心功能
1	仪表盘	Dashboard	账户资产总览、持仓状态、AI 信号、市场动态
2	回测系统	Backtest	策略回测配置与执行、结果展示、风险分析
3	交易雷达	Radar	AI 智能扫描、信号检测、市场情绪分析
4	策略管理	Strategies	策略 CRUD、启停控制、AI 优化建议
5	实时交易	Trading	K 线图表、下单交易、订单簿、持仓管理
6	交易历史	History	历史记录查询、筛选导出、收益曲线
7	账户中心	Account	用户信息、安全设置、API 密钥、风控、订阅

2 技术栈选型

2.1 前端技术栈

技术项	选型	版本	用途说明
框架	React	18.x	组件化开发，生态成熟
语言	TypeScript	5.x	类型安全，提升开发效率
构建工具	Vite	5.x	极速构建，HMR 热更新
UI 组件库	Ant Design	5.x	丰富的中后台组件
状态管理	Zustand	4.x	轻量级状态管理
图表库	ECharts	5.x	K 线图、雷达图、曲线图等
路由	React Router	6.x	前端路由管理

HTTP 客户端	Axios	1.x	API 请求
WebSocket	Socket.io-client	4.x	实时行情推送
CSS 方案	Tailwind CSS	3.x	原子化 CSS，深色主题定制
图标	Lucide React	0.x	统一的线性风格图标
数字字体	JetBrains Mono	-	等宽字体，用于金融数据

2.2 后端技术栈

技术项	选型	版本	用途说明
语言	Python	3.11+	主开发语言，量化生态原生支持
Web 框架	FastAPI	0.110+	高性能异步框架，自动 API 文档
ORM	SQLAlchemy	2.x	数据库 ORM，支持异步操作
数据验证	Pydantic	2.x	请求/响应数据验证
任务队列	Celery + Redis	-	异步任务处理
WebSocket	python-socketio	5.x	实时行情推送
认证	JWT (python-jose)	-	用户认证与授权
日志	Loguru	-	结构化日志

2.3 数据层技术栈

技术项	选型	用途说明
关系型数据库	PostgreSQL 16	用户、策略、订单、交易记录等核心数据
时序数据库	TimescaleDB	行情数据、K 线数据、资金曲线等时间序列
缓存	Redis 7	会话、实时行情缓存、任务队列、限流
消息队列	Redis Streams	行情数据流、交易信号流
文件存储	MinIO	回测报告、导出文件存储

2.4 量化/AI 技术栈

技术项	选型	用途说明
数据处理	Pandas + NumPy	时间序列分析、数值计算
技术指标	TA-Lib	移动平均线、MACD、RSI、布林带等
回测引擎	Backtrader	策略回测框架
机器学习	Scikit-learn	信号分类、回归预测
AI 信号引擎	自研模块	基于技术指标+ML 的交易信号识别

2.5 部署与运维

技术项	选型	用途说明
容器化	Docker + Docker Compose	开发/生产环境一致
反向代理	Nginx	静态资源服务、负载均衡
进程管理	Gunicorn + Uvicorn	后端服务进程管理
CI/CD	GitHub Actions	自动化测试、构建、部署

监控	Prometheus + Grafana	服务监控、告警
----	----------------------	---------

3 系统架构设计

3.1 整体架构图

系统采用前后端分离架构，分为表现层、接口层、业务逻辑层、数据访问层和数据存储层。



3.2 前端项目结构

```
futurespro-frontend/
src/
  components/      # 公共组件
    layout/        # 布局 (Header, Sidebar)
    charts/        # 图表 (KLine, Radar, Curve)
    trade/         # 交易 (OrderPanel, OrderBook)
    ui/            # 基础UI (Card, Modal, Badge)
  pages/          # 页面组件
    Dashboard/     # 仪表盘
    Trading/       # 实时交易
    Backtest/      # 回测系统
    Radar/         # 交易雷达
    Strategies/    # 策略管理
    History/       # 交易历史
    Account/       # 账户中心
  hooks/          # 自定义 Hooks
  stores/         # Zustand 状态管理
  services/       # API 服务层
  types/          # TypeScript 类型
  utils/          # 工具函数
  styles/         # 全局样式、主题变量
```

3.3 后端项目结构

```
futurespro-backend/
├── app/
│   ├── api/v1/
│   │   ├── # API 路由
│   │   ├── auth.py           # 认证接口
│   │   ├── dashboard.py      # 仪表盘接口
│   │   ├── trading.py        # 交易接口
│   │   ├── backtest.py       # 回测接口
│   │   ├── radar.py          # 雷达接口
│   │   ├── strategies.py     # 策略接口
│   │   ├── history.py        # 历史接口
│   │   ├── account.py        # 账户接口
│   │   └── market.py         # 行情接口
│   ├── core/                 # 核心配置
│   ├── models/               # 数据库模型
│   ├── schemas/              # Pydantic 模式
│   ├── services/             # 业务逻辑层
│   ├── tasks/                # Celery 异步任务
│   ├── websocket/            # WebSocket 处理
│   ├── main.py               # 应用入口
│   ├── migrations/           # 数据库迁移
│   ├── tests/                # 测试
│   ├── requirements.txt
│   ├── Dockerfile
│   └── docker-compose.yml
```

4 数据库设计

4.1 核心数据表

系统共设计 12 张核心数据表，覆盖用户、交易、策略、行情等全部业务领域。

4.1.1 用户表 (users)

字段名	类型	约束	说明
id	UUID	PK	用户 ID
username	VARCHAR(50)	UNIQUE	用户名
email	VARCHAR(100)	UNIQUE	邮箱
password_hash	VARCHAR(255)	NOT NULL	密码哈希
role	ENUM	DEFAULT user	角色
plan	ENUM	DEFAULT free	订阅计划
two_factor_enabled	BOOLEAN	DEFAULT false	二步验证
created_at	TIMESTAMP	DEFAULT NOW()	创建时间

4.1.2 账户表 (accounts)

字段名	类型	约束	说明
id	UUID	PK	账户 ID
user_id	UUID	FK users	关联用户
balance	DECIMAL(18,2)	DEFAULT 0	账户余额
frozen_margin	DECIMAL(18,2)	DEFAULT 0	已用保证金
total_pnl	DECIMAL(18,2)	DEFAULT 0	累计盈亏
today_pnl	DECIMAL(18,2)	DEFAULT 0	今日盈亏
win_rate	DECIMAL(5,4)	DEFAULT 0	胜率
total_trades	INTEGER	DEFAULT 0	累计交易次数

4.1.3 合约表 (contracts)

字段名	类型	约束	说明
id	UUID	PK	合约 ID
symbol	VARCHAR(20)	UNIQUE	合约代码
name	VARCHAR(50)	NOT NULL	合约名称
exchange	VARCHAR(20)	NOT NULL	交易所
multiplier	DECIMAL(10,2)	NOT NULL	合约乘数
margin_rate	DECIMAL(5,4)	NOT NULL	保证金比例

4.1.4 订单表 (orders)

字段名	类型	约束	说明
id	UUID	PK	订单 ID
user_id	UUID	FK users	用户
contract_id	UUID	FK contracts	合约
strategy_id	UUID	FK strategies	关联策略
direction	ENUM	NOT NULL	方向(long/short)
order_type	ENUM	NOT NULL	类型(market/limit/stop)
quantity	INTEGER	NOT NULL	数量
price	DECIMAL(18,2)		现价
leverage	INTEGER	DEFAULT 1	杠杆
status	ENUM	DEFAULT pending	状态
commission	DECIMAL(18,2)		手续费

4.1.5 持仓表 (positions)

字段名	类型	约束	说明
id	UUID	PK	持仓 ID
user_id	UUID	FK users	用户
contract_id	UUID	FK contracts	合约
direction	ENUM	NOT NULL	方向
quantity	INTEGER	NOT NULL	持仓数量

avg_price	DECIMAL(18,2)	NOT NULL	开仓均价
current_price	DECIMAL(18,2)		当前价
floating_pnl	DECIMAL(18,2)		浮动盈亏
margin	DECIMAL(18,2)		占用保证金

4.1.6 交易记录表 (trade_history)

字段名	类型	约束	说明
id	UUID	PK	记录 ID
user_id	UUID	FK users	用户
contract_id	UUID	FK contracts	合约
direction	ENUM	NOT NULL	方向
open_price	DECIMAL(18,2)	NOT NULL	开仓价
close_price	DECIMAL(18,2)		平仓价
quantity	INTEGER	NOT NULL	数量
pnl	DECIMAL(18,2)		盈亏
status	ENUM	DEFAULT open	状态(open/closed)

4.1.7 策略表 (strategies)

字段名	类型	约束	说明
id	UUID	PK	策略 ID
user_id	UUID	FK users	创建者
name	VARCHAR(100)	NOT NULL	策略名称
type	ENUM	NOT NULL	类型(trend/breakout...)
params	JSONB	NOT NULL	策略参数(JSON)
status	ENUM	DEFAULT stopped	状态(running/stopped)
total_return	DECIMAL(10,4)	DEFAULT 0	总收益率
sharpe_ratio	DECIMAL(6,2)	DEFAULT 0	夏普比率
max_drawdown	DECIMAL(10,4)	DEFAULT 0	最大回撤
win_rate	DECIMAL(5,4)	DEFAULT 0	胜率

4.1.8 回测记录表 (backtests)

字段名	类型	约束	说明
id	UUID	PK	回测 ID
strategy_id	UUID	FK strategies	关联策略
contract_id	UUID	FK contracts	合约
start_date	DATE	NOT NULL	回测开始
end_date	DATE	NOT NULL	回测结束
initial_capital	DECIMAL(18,2)	NOT NULL	初始资金
params	JSONB	NOT NULL	回测参数
total_return	DECIMAL(10,4)		总收益率
monthly_returns	JSONB		月度收益(JSON)

status	ENUM	DEFAULT pending	状态
--------	------	-----------------	----

4.1.9 AI 信号表 (ai_signals)

字段名	类型	约束	说明
id	UUID	PK	信号 ID
contract_id	UUID	FK contracts	合约
signal_type	ENUM	NOT NULL	信号类型
direction	ENUM	NOT NULL	方向
confidence	DECIMAL(5,2)	NOT NULL	置信度(0-100)
entry_price	DECIMAL(18,2)		入场价
target_price	DECIMAL(18,2)		目标价
stop_price	DECIMAL(18,2)		止损价
status	ENUM	DEFAULT active	状态

4.1.10 监控规则表 (monitor_rules)

字段名	类型	约束	说明
id	UUID	PK	规则 ID
user_id	UUID	FK users	用户
name	VARCHAR(100)	NOT NULL	规则名称
rule_type	ENUM	NOT NULL	类型
params	JSONB	NOT NULL	规则参数
is_active	BOOLEAN	DEFAULT true	是否启用

4.1.11 API 密钥表 (api_keys)

字段名	类型	约束	说明
id	UUID	PK	密钥 ID
user_id	UUID	FK users	用户
name	VARCHAR(100)	NOT NULL	密钥名称
key_id	VARCHAR(50)	UNIQUE	密钥 ID
key_secret_hash	VARCHAR(255)	NOT NULL	密钥哈希
environment	ENUM	DEFAULT production	环境
permissions	JSONB	NOT NULL	权限范围
is_active	BOOLEAN	DEFAULT true	是否活跃

4.1.12 风控设置表 (risk_settings)

字段名	类型	约束	说明
id	UUID	PK	设置 ID
user_id	UUID	FK users UNIQUE	用户(一人一份)
max_daily_loss	DECIMAL(18,2)	DEFAULT 50000	单日最大亏损
max_single_loss	DECIMAL(18,2)	DEFAULT 20000	单笔最大亏损
max_position_size	INTEGER	DEFAULT 20	最大持仓手数

auto_risk_control	BOOLEAN	DEFAULT true	自动风控
risk_level	ENUM	DEFAULT medium	风险等级

4.2 时序数据表 (TimescaleDB)

4.2.1 行情数据表 (market_quotes)

字段名	类型	说明
time	TIMESTAMPTZ	时间戳(联合主键)
contract_id	UUID	合约 ID(联合主键)
open	DECIMAL(18,2)	开盘价
high	DECIMAL(18,2)	最高价
low	DECIMAL(18,2)	最低价
close	DECIMAL(18,2)	收盘价
volume	BIGINT	成交量
open_interest	BIGINT	持仓量

4.2.2 K 线数据表 (kline_data)

字段名	类型	说明
time	TIMESTAMPTZ	时间戳(联合主键)
contract_id	UUID	合约 ID(联合主键)
period	VARCHAR(10)	周期(1m/5m/15m/1h/1d)
open/high/low/close	DECIMAL(18,2)	OHLC
volume	BIGINT	成交量

5 API 接口设计

5.1 认证接口

方法	路径	说明
POST	/api/v1/auth/register	用户注册
POST	/api/v1/auth/login	用户登录(返回 JWT)
POST	/api/v1/auth/refresh	刷新 Token
POST	/api/v1/auth/logout	用户登出
POST	/api/v1/auth/change-password	修改密码
POST	/api/v1/auth/2fa/enable	开启二步验证

5.2 仪表盘接口

方法	路径	说明
GET	/api/v1/dashboard/stats	获取账户统计
GET	/api/v1/dashboard/positions	获取当前持仓

GET	/api/v1/dashboard/market	获取市场动态
GET	/api/v1/dashboard/ai-signals	获取 AI 信号

5.3 交易接口

方法	路径	说明
GET	/api/v1/quotes	获取实时行情
GET	/api/v1/klines/{symbol}/{period}	获取 K 线数据
GET	/api/v1/orderbook/{symbol}	获取订单簿
POST	/api/v1/orders	提交订单
POST	/api/v1/positions/{id}/close	平仓
POST	/api/v1/positions/close-all	批量平仓
GET	/api/v1/contracts	获取合约列表

5.4 回测系统接口

方法	路径	说明
POST	/api/v1/backtest/run	运行回测(异步)
GET	/api/v1/backtest/{id}/report	获取回测报告
GET	/api/v1/backtest/{id}/trades	获取交易记录
GET	/api/v1/backtest/{id}/monthly	获取月度收益
POST	/api/v1/backtest/batch	批量回测
POST	/api/v1/backtest/optimize	参数优化
GET	/api/v1/backtest/export/{id}	导出报告

5.5 交易雷达接口

方法	路径	说明
GET	/api/v1/radar/signals	获取交易信号
GET	/api/v1/radar/scan/status	AI 扫描状态
POST	/api/v1/radar/scan/toggle	开关 AI 扫描
GET	/api/v1/radar/market-strength	市场强度
GET	/api/v1/radar/sentiment	市场情绪
GET	/api/v1/radar/patterns	形态检测
POST	/api/v1/radar/rules	添加监控规则
GET	/api/v1/radar/export	导出信号

5.6 策略管理接口

方法	路径	说明
GET	/api/v1/strategies	获取策略列表
POST	/api/v1/strategies	创建策略
PUT	/api/v1/strategies/{id}	更新策略

DELETE	/api/v1/strategies/{id}	删除策略
POST	/api/v1/strategies/{id}/start	启动策略
POST	/api/v1/strategies/{id}/stop	停止策略
GET	/api/v1/ai/suggestions	AI 优化建议

5.7 交易历史接口

方法	路径	说明
GET	/api/v1/history	获取交易历史
GET	/api/v1/history/{id}	获取交易详情
GET	/api/v1/history/stats	获取交易统计
POST	/api/v1/history/export	导出记录
POST	/api/v1/history/sync	同步交易数据

5.8 账户中心接口

方法	路径	说明
GET/PUT	/api/v1/user/profile	获取/更新用户信息
GET	/api/v1/user/assets	获取资产概览
GET/POST	/api/v1/api-keys	获取/创建 API 密钥
GET/PUT	/api/v1/risk/settings	风控设置
GET	/api/v1/risk/report	风险评估报告
GET/PUT	/api/v1/notifications/settings	通知设置
GET	/api/v1/subscription	订阅信息

5.9 WebSocket 接口

事件名	方向	说明
quote:update	Server->Client	实时行情推送
kline:update	Server->Client	K 线数据更新
orderbook:update	Server->Client	订单簿更新
position:update	Server->Client	持仓变动通知
order:filled	Server->Client	订单成交通知
signal:new	Server->Client	新 AI 信号推送
backtest:progress	Server->Client	回测进度推送
subscribe:quotes	Client->Server	订阅行情
subscribe:kline	Client->Server	订阅 K 线
unsubscribe	Client->Server	取消订阅

6 页面实现方案

6.1 公共组件

组件名称	说明	复用页面
AppLayout	全局布局(头部+侧边栏+内容区)	所有页面
AppHeader	顶部导航栏(Logo/导航/搜索/通知/用户)	所有页面
DataCard	数据展示卡片(标签/数值/变化量)	仪表盘/回测/策略
StatsCard	统计指标卡片	仪表盘/回测/策略
Modal	通用模态框组件	所有页面(21 个弹窗)
ConfirmDialog	确认对话框	平仓/删除/批量操作
Badge	状态标签(运行中/停止/做多/做空)	所有页面
DataTable	数据表格(支持排序/筛选/分页)	交易/历史/策略
ToggleButton	开关切换组件	账户/雷达
RangeSlider	滑块组件	交易/回测/账户
SearchInput	搜索输入框	导航/交易/历史
TabNav	标签页导航	策略/历史/交易
FilterBar	筛选工具栏	历史/交易/雷达
Pagination	分页器	历史/策略
NotificationBell	通知铃铛组件	导航栏

6.2 图表组件

组件名称	图表类型	技术方案	使用页面
KLineChart	K 线图(蜡烛图)	ECharts candlestick	实时交易
EquityCurve	资金曲线图	ECharts line+area	回测/历史
RadarChart	市场强度雷达图	ECharts radar	交易雷达
DrawdownChart	回撤曲线图	ECharts area	回测
PieChart	交易分布饼图	ECharts pie/doughnut	历史
MiniCurve	迷你收益曲线	ECharts mini line	策略卡片
ProgressBar	进度条/仪表条	CSS+SVG	雷达/回测/账户
SentimentBar	市场情绪双色条	CSS	交易雷达
MonthlyGrid	月度收益网格	CSS Grid	回测

6.3 各页面实现详细

6.3.1 仪表盘页面 (Dashboard)

页面布局：上方统计卡片区(4 卡) + 快捷操作按钮行 + 下方左右两栏布局

- 统计卡片区：总资产、今日盈亏、当日最大回撤、今日交易次数，WebSocket 实时更新
- 快捷操作按钮：快速下单、持仓详情、策略管理、交易记录
- 持仓表格：合约/方向/持仓量/开仓价/当前价/浮动盈亏/操作

4. AI 智能信号面板：展示 AI 识别的交易信号
5. 市场动态列表：实时行情 WebSocket 更新
6. 弹窗(4 个)：快速下单、平仓确认、批量平仓、合约选择

6.3.2 实时交易页面 (Trading)

页面布局：三栏布局（左侧合约列表 | 中间图表+下单 | 右侧订单簿+持仓）

1. 左侧合约面板：实时行情列表，带筛选标签，点击切换当前合约
2. K 线图表：ECharts candlestick，支持分时/1 分/5 分/日线切换，叠加 MA5/MA10/MA20
3. 下单面板：市价/限价/止损/条件单标签切换，含杠杆滑块、预估保证金
4. 订单簿：价格/成交量/持仓量三列展示，WebSocket 实时更新
5. 弹窗(3 个)：订单确认、平仓确认、合约详情

6.3.3 回测系统页面 (Backtest)

页面布局：左侧边栏(策略列表+统计+工具) + 右侧主内容区

1. 左侧边栏：我的策略列表、快速统计、常用工具(批量回测/导出报告/参数优化)
2. 工具栏：策略选择、品种选择、时间范围、周期选择、运行回测按钮
3. 核心指标卡片(4 个)：总收益率、夏普比率、最大回撤、胜率
4. 资金曲线图 + 交易记录列表 + 风险分析面板 + 月度收益网格
5. AI 建议面板：基于历史数据的策略优化建议
6. 弹窗(4 个)：回测报告详情、批量回测、导出报告、参数优化

6.3.4 交易雷达页面 (Radar)

页面布局：左侧面板(AI 扫描+行情+规则) + 右侧主内容区

1. AI 智能扫描卡片：脉冲动画图标、实时扫描 Toggle、进度条、发现机会数
2. 市场强度雷达图：六维雷达图(趋势/动量/波动/情绪/流动性/成交量)
3. 交易机会卡片(4 张)：合约/方向/来源/置信度
4. 市场情绪分析：双色进度条展示多空比例
5. 形态检测列表：双底/均线多头/MACD 死叉/矩形整理等
6. 弹窗(3 个)：信号详情、添加规则、导出信号

6.3.5 策略管理页面 (Strategies)

页面布局：上方统计行 + 标签页切换 + 策略卡片网格 + AI 建议面板

1. 统计行(4 个)：运行中策略数、今日收益、平均收益率、累计交易
2. 标签页：全部策略/运行中/已停止/优化中
3. 策略卡片：策略名、类型标签、状态指示灯、收益率/夏普/胜率/回撤、迷你曲线、操作按钮
4. AI 策略优化建议面板：3 张建议卡片，含置信度和应用按钮
5. 弹窗(5 个)：创建策略、策略详情、删除确认、批量回测、AI 建议详情

6.3.6 交易历史页面 (History)

页面布局：上方图表区(收益曲线+分布饼图) + 下方筛选工具栏 + 交易记录表格

1. 收益曲线图：带时间范围选择(近 7 天/30 天/90 天/全部)
2. 交易分布饼图：胜率、盈利、亏损、持平占比
3. 筛选工具栏：合约/方向/状态/日期范围/搜索
4. 交易记录表格：时间/合约/方向/类型/手数/开仓价/平仓价/盈亏/策略/状态/操作
5. 分页器：支持页码跳转
6. 弹窗(3 个)：交易详情、导出记录、同步数据

6.3.7 账户中心页面 (Account)

页面布局：用户信息卡片 + 资产卡片区 + 设置区(左侧导航+右侧内容)

1. 用户信息卡片：头像、用户名、邮箱、VIP/Pro/已认证徽章、交易天数/累计收益/胜率
2. 资产卡片(3 个)：账户余额、浮动盈亏、已用保证金
3. 设置导航(6 项)：基本信息、安全设置、API 密钥、风险控制、通知设置、账单与订阅
4. 风险控制：风险等级指示、单日/单笔最大亏损滑块、自动风控 Toggle、冷静期设置
5. API 密钥管理：密钥列表、创建/重新生成/删除操作
6. 弹窗(4 个)：修改密码、创建 API 密钥、风险评估报告、升级订阅

7 核心业务逻辑设计

7.1 交易执行流程

1. 用户提交订单 -> 风控引擎检查(保证金/持仓限制/亏损限制) -> 订单入库(pending)
2. 订单匹配引擎处理 -> 更新订单状态(filled) -> 更新持仓表
3. 计算浮动盈亏 -> WebSocket 推送持仓变动 -> 更新账户余额
4. 触发风控检查(单日亏损/单笔亏损) -> 必要时触发强制平仓

7.2 策略引擎流程

1. 策略启动 -> 加载策略参数和关联合约 -> 注册行情订阅
2. 接收实时行情数据 -> 计算技术指标 -> 触发交易信号
3. 信号通过风控检查 -> 自动下单 -> 记录交易日志
4. 定期计算策略绩效指标 -> 更新策略统计

7.3 回测引擎流程

1. 用户配置回测参数(策略/合约/时间范围/资金) -> 创建回测任务
2. Celery 异步执行 -> 加载历史 K 线数据 -> 逐 Bar 模拟交易
3. 计算每笔交易盈亏 -> 生成资金曲线 -> 计算绩效指标
4. 生成月度收益网格 -> 风险分析 -> 保存回测报告

- 5. WebSocket 推送回测进度和完成通知

7.4 AI 信号引擎流程

- 1. 定时任务扫描所有活跃合约 -> 计算技术指标(MA/MACD/RSI/布林带)
- 2. 形态识别(双底/双顶/头肩/三角形) -> 信号评分
- 3. ML 模型预测(基于历史特征) -> 综合置信度计算
- 4. 生成 AI 信号 -> 存入数据库 -> WebSocket 推送
- 5. 用户监控规则匹配 -> 触发自定义通知

7.5 风控引擎流程

- 1. 实时监控：持仓盈亏、保证金使用率、单日亏损累计
- 2. 触发条件：单笔亏损超阈值->警告/自动平仓，单日亏损超阈值->禁止新开仓/强制冷静期
- 3. 保证金使用率超过 90% -> 强制减仓
- 4. 风险评估报告：VaR 计算、最大回撤分析、夏普比率计算

8 实施计划

8.1 第一阶段：基础架构搭建（2 周）

任务	工期	产出
前端项目初始化(Vite+React+TS+Tailwind)	3 天	可运行的前端框架
后端项目初始化(FastAPI+SQLAlchemy)	3 天	可运行的后端框架
数据库设计与迁移	2 天	完整的数据库 Schema
Docker Compose 环境搭建	1 天	一键启动的开发环境
公共组件开发(15 个)	5 天	全站可复用组件库

8.2 第二阶段：核心交易功能（3 周）

任务	工期	产出
实时交易页面(K 线图+下单+订单簿)	5 天	完整的交易界面
交易 API 开发(下单/平仓/撤单)	4 天	交易 REST API
WebSocket 实时行情推送	3 天	实时数据流
持仓管理	2 天	持仓 CRUD
风控引擎基础版	3 天	基本风控能力
仪表盘页面	3 天	数据总览页面

8.3 第三阶段：量化研究功能（3 周）

任务	工期	产出
回测引擎开发	5 天	策略回测能力

回测系统页面	4 天	回测配置与结果展示
策略引擎开发	4 天	自动交易策略
策略管理页面	3 天	策略 CRUD 界面
AI 信号引擎	4 天	信号识别能力
交易雷达页面	3 天	信号展示界面

8.4 第四阶段：辅助功能完善（2 周）

任务	工期	产出
交易历史页面	3 天	历史记录查询
账户中心页面	4 天	设置管理界面
导出功能(CSV/Excel/PDF)	2 天	数据导出
通知系统	2 天	消息推送
数据同步功能	1 天	云端同步

8.5 第五阶段：测试与优化（2 周）

任务	工期	产出
单元测试	3 天	核心模块测试覆盖
集成测试	3 天	API 端到端测试
性能优化	2 天	前后端性能调优
安全审计	1 天	安全漏洞检查
部署上线	1 天	生产环境部署

总工期估算：约 12 周（3 个月）

9 非功能性需求

9.1 性能要求

- 1. API 响应时间 < 200ms (P95)
- 2. WebSocket 延迟 < 100ms
- 3. K 线图渲染帧率 > 30fps
- 4. 支持 1000+ 并发用户

9.2 安全要求

- 1. JWT Token 认证，Token 过期时间 24h
- 2. 密码 bcrypt 加密存储
- 3. API 密钥 HMAC-SHA256 签名
- 4. HTTPS 全链路加密
- 5. SQL 注入防护（ORM 参数化查询）
- 6. XSS 防护（输入过滤+输出编码）

7. 速率限制 (100 次/分钟/IP)

9.3 可用性要求

1. 系统可用性 > 99.9%
2. 数据库主从备份
3. Redis 持久化 (AOF)
4. 定时数据备份 (每日)
5. 异常监控与告警

9.4 可扩展性

1. 水平扩展: 无状态 API 服务, 支持多实例部署
2. 数据分片: 时序数据按合约+时间分片
3. 缓存策略: 多级缓存 (本地+Redis)
4. 异步处理: 耗时操作全部异步化